

基于 PagedAttention 的大语言模型服务高效内存管理

(vLLM 论文全文中文翻译)

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, Ion Stoica
UC Berkeley / Stanford / UC San Diego
SOSP'23 | arXiv:2309.06180

摘要

大语言模型 (LLM) 的高吞吐服务需要同时对大量请求进行批处理。然而，现有系统面临困难，因为每个请求的键值缓存 (KV cache) 占用大量内存且动态增缩。当内存管理不善时，碎片化和冗余复制会造成显著浪费，限制了批处理大小。为此，我们提出 PagedAttention —— 一种受经典操作系统虚拟内存与分页技术启发的注意力算法。在此基础上，我们构建了 vLLM，一个 LLM 服务系统，实现了 (1) KV cache 内存接近零浪费，以及 (2) 在请求内部和请求之间灵活共享 KV cache 以进一步减少内存使用。评估表明，vLLM 在相同延迟水平下将主流 LLM 的吞吐量提升 2-4 倍 (对比 FasterTransformer 和 Orca 等当时最先进系统)。序列越长、模型越大、解码算法越复杂时，改进越显著。

1. 引言

GPT、PaLM 等大语言模型催生了编程助手、通用聊天机器人等新应用，云厂商争相提供托管服务。然而运行这些应用代价极高——处理一次 LLM 请求的成本是传统关键词检索的 10 倍。因此提升 LLM 服务系统的吞吐量 (从而降低单请求成本) 变得至关重要。

LLM 的核心是自回归 Transformer 模型，每次生成一个 token，直到产生终止符。这种顺序生成过程使得工作负载是内存瓶颈型 (memory-bound)，GPU 算力利用不足。可以通过批处理多个请求来提升吞吐，但前提是每个请求的内存空间要被高效管理。

以 13B 参数模型在 A100 40GB 上为例：模型权重约占 65% (26GB)，KV cache 动态占用 >30%，剩余少量用于激活值。KV cache 管理效率直接决定了最大批处理大小。

现有系统的两个核心问题：

问题一：内存碎片化。现有系统将一个请求的 KV cache 存储在连续内存空间中，必须按最大长度 (如 2048) 预分配。实际长度通常远小于最大长度，导致严重的内部碎片。同时不同请求预分配的大小不同，释放后留下大小不一的空隙 (外部碎片)。实测仅 20.4%-38.2% 的 KV cache 内存用于存储实际 token 状态。

问题二：无法共享内存。并行采样、Beam Search 等解码算法会为一个请求生成多个输出序列，这些序列可以部分共享 KV cache (共同的前缀部分)。但现有系统因为每个序列的 KV cache 存储在独立的连续空间中，无法实现内存共享。

我们的贡献：

- 识别 LLM 服务中内存分配的挑战并量化其对性能的影响
- 提出 PagedAttention，受 OS 虚拟内存/分页启发，在非连续分页内存上运行的注意力算法
- 设计并实现 vLLM，基于 PagedAttention 的高吞吐分布式 LLM 服务引擎
- 在多种场景下展示 vLLM 大幅优于 FasterTransformer 和 Orca

2. 背景

2.1 基于 Transformer 的大语言模型

语言模型的任务是对 token 序列的概率分布建模： $P(x) = P(x_1) \cdot P(x_2|x_1) \cdot \dots \cdot P(x_n|x_1, \dots, x_{n-1})$ 。当前最成功的方法是 Transformer 架构，核心组件是自注意力层。输入序列的每个 token 被映射为 Query (Q)、Key (K)、Value (V) 向量，注意力输出为 $\text{softmax}(QK^T/\text{sqrt}(d)) \cdot V$ 。

LLM 推理分两个阶段：(1) Prompt 阶段 (Prefill)：将用户完整 prompt 一次性输入，并行计算所有位置的 KV；(2) 自回归生成阶段 (Decode)：每次只输入前一步生成的 token，计算新的 KV 并与历史 KV 拼接后做 attention。为避免重复计算，所有历史 token 的 Key 和 Value 被缓存下来，称为 KV cache。

2.2 LLM 服务与迭代级调度

LLM 服务场景下，多个用户请求并发到达。由于请求的输入/输出长度各异、到达时间不同，简单的请求级批处理（一批请求全部完成才开始下一批）效率低下。Orca 提出迭代级调度 (iteration-level scheduling)：每生成一个 token 就重新决定 batch 组成——已完成的请求移出、新请求加入。这为动态批处理奠定了基础。

3. LLM 服务中的内存管理挑战

KV cache 有三个独特特征使其难以高效管理：

- (1) 大：单个 token 的 KV cache 在 LLaMA-13B 中约为数百 KB。一个序列（如 2048 token）的 KV cache 可达 GB 级。
- (2) 动态：长度随生成过程增长，事先不可预知。
- (3) 生命周期不确定：请求何时结束取决于模型输出。

三种内存浪费：

- ① 预留浪费 (Reserved waste)：为每个请求按 max_length 预分配连续空间，但多数请求实际使用远少于于此。
- ② 内部碎片 (Internal fragmentation)：预分配槽位中，已分配但当前未使用的部分。
- ③ 外部碎片 (External fragmentation)：请求结束后释放的内存块大小不一，新请求无法找到足够大的连续空间。

实验测量结果 (图2)：在 Orca(Max) 策略下，实际有效利用率仅 20.4%；Orca(Pow2) 为 26.8%；即使使用不可能实现的 Oracle 策略（事先知道每个请求长度），有效利用率也只有 57.3%（仍有外部碎片）。vLLM 达到 96.3%。

4. PagedAttention 方法

4.1 PagedAttention 算法

核心理念：将每个序列的 KV cache 划分为固定大小的 KV block，每个 block 存储固定数量 token（如 16 个）的 Key 和 Value 向量。与传统方案不同，这些 block 在物理内存中不必连续。

类比操作系统：block 对应物理页（page），token 对应字节（byte），序列对应进程（process）。每个序列维护一张 block table（对应 OS 页表），记录逻辑 block 到物理 block 的映射。

在计算 attention 时，PagedAttention kernel 根据 block table 从不连续的物理地址获取 K、V 数据，然后正常执行注意力计算。这是一个 fused kernel —— 将间接寻址和 attention 计算融合到一个 GPU kernel 中。

PagedAttention 伪码核心逻辑：

对于给定的 query 向量 q （当前 token 的 Q ）：

1. 遍历 block table 中所有已分配的物理 block
2. 对每个 block，从物理地址加载 K 向量，计算 $q \cdot k^T / \sqrt{d}$
3. 对所有 block 的 score 做全局 softmax
4. 用 softmax 权重对所有 block 的 V 向量加权求和

整个过程与标准 attention 数学等价，只是数据来源从连续内存变为分散 block。

4.2 KV Cache Manager

vLLM 维护一个全局 block allocator（块分配器），类似 OS 的物理页分配器。管理机制：

- 分配：新 token 生成时，检查当前序列最后一个 block 是否有空位。有则直接写入；无则从全局 pool 申请一个新的物理 block。
- 释放：序列完成后，其 block table 中所有物理 block 归还到全局 pool。
- 优势：只有最后一个 block 可能有内部碎片（最多浪费 $\text{block_size} - 1$ 个 token 的空间）；完全没有外部碎片（所有 block 大小相同）。

4.3 其他解码场景的应用

PagedAttention 的 block 粒度管理使得以下高级场景变得高效：

并行采样（Parallel Sampling）

同一个 prompt 生成多个回答时，所有样本共享 prompt 部分的 KV block。通过引用计数（reference counting）管理共享 block 的生命周期。只有各样本各自生成的后续 token 才需要独立 block。共享 prompt KV 使内存节省高达 50-80%（取决于 prompt 占总长度的比例）。

Beam Search

Beam Search 中多个候选序列（beam）共享不同长度的公共前缀。vLLM 使用 Copy-on-Write (CoW) 机制：

- 多个 beam 共享相同的 block（引用计数 > 1 ）

- 当某个 beam 要修改一个共享 block 时 (向已满的 block 追加 token) , 才真正复制一份物理 block 这与 Unix fork() 的 CoW 语义完全一致。传统实现中 Beam Search 需要为每个 beam 完整复制所有 KV cache , PagedAttention 的 CoW 大幅减少了复制开销。

共享前缀 (Shared Prefix / System Prompt)

多用户共享相同 system prompt 时, 该前缀的 KV block 可以被所有请求复用。实现方式: 全局维护一个前缀 KV 的 block 集合, 新请求的 block table 前几项直接指向已有的物理 block, 无需重新计算。

5. 调度与抢占

5.1 调度策略

vLLM 的调度器与 KV cache manager 协同工作:

- 使用先来先服务 (FCFS) 作为基本调度策略
- 配合迭代级调度 (继承自 Orca) : 每完成一个 decode step 就重新评估 batch 组成
- 连续批处理 (Continuous Batching) : 某个请求生成 EOS 后立刻移出, 新请求即刻加入当前 batch , GPU 不空转

5.2 抢占机制 (Preemption)

当 GPU 显存不足以容纳所有活跃请求的 KV cache 时, vLLM 需要抢占一部分请求。两种策略:

- ① 交换 (Swapping) : 将被抢占请求的 KV cache block 从 GPU 复制到 CPU 内存。当 GPU 显存空闲后再换回。适合 KV cache 较小、CPU 内存充裕的场景。
- ② 重计算 (Recomputation) : 直接丢弃被抢占请求的 KV cache , 等有资源后重新跑一次 Prefill。适合 prompt 较短 (重算代价低) 、CPU 内存紧张的场景。

抢占的优先级策略: vLLM 优先抢占最后到达的请求 (避免饥饿) , 并在 block 级别进行精确的内存回收。

6. 实现

vLLM 实现为一个端到端的 LLM 服务系统, 由以下组件构成:

- Centralized Scheduler : 中心化调度器运行在 CPU 上, 管理请求队列和 KV cache 分配
- KV Cache Manager : block 分配器 + block table 管理
- Worker : 每个 GPU 上运行一个 worker , 执行模型前向计算
- PagedAttention Kernel : 自定义 CUDA kernel , 支持从不连续 block 读取 K/V 并计算 attention
- 分布式支持 : 通过 Megatron-LM 风格的张量并行 + Ray 实现多 GPU/多机推理

支持模型: GPT-2、GPT-J、OPT (125M~175B) 、LLaMA (7B~65B) 等。默认 block_size = 16 token。

7. 实验评估

7.1 实验设置

- 模型：OPT-13B、OPT-175B、LLaMA-13B
- 硬件：NVIDIA A100 80GB
- 基线：FasterTransformer (NVIDIA 官方)、Orca (迭代级调度最先进系统)
- 指标：吞吐量 (tokens/s)、延迟 (normalized latency)
- 工作负载：ShareGPT (真实对话，长输出) 和 Alpaca (短输出) 数据集

7.2 主要结果

吞吐量对比 (核心数据)：

- vs FasterTransformer：vLLM 吞吐量高 14-24 倍
- vs Orca (最优配置)：vLLM 吞吐量高 2.2-4.3 倍
- KV cache 有效利用率：vLLM 达 96.3% (对比 Orca 的 20.4-38.2%)

在 ShareGPT 数据集 (长序列) 上改进更为显著，因为序列越长 KV cache 占用越多，内存管理效率的差距越容易暴露。

7.3 消融实验

Block size 选择：测试了 block_size = 1, 2, 4, 8, 16, 32, 64, 128。结果 16 和 32 效果最好——太小则 block table 开销大、kernel 效率低；太大则内部碎片增多。论文选定默认值 16。

Kernel 性能：PagedAttention kernel 相比标准连续 attention kernel 有轻微开销 (间接寻址代价)，但在实际端到端场景下该开销可忽略 (<5%)。

7.4 并行采样与 Beam Search

- 并行采样：共享 prompt KV 使得相同吞吐下内存占用减少 50%+
- Beam Search (beam width=6)：CoW 机制下 vLLM 吞吐为 Orca 的 2.3 倍

8. 讨论

适用范围与局限

- PagedAttention 对模型精度无任何影响 (数学等价，只改变内存布局)
- 主要局限：自定义 kernel 开发成本高，需要为每种新的 attention 变体重新适配
- block_size 是固定的超参数——未来可探索自适应 block size
- 当前实现假设单个 KV head 的 head_dim 能整除 block_size 中的元素数，不满足时需要 padding

与 OS 虚拟内存的深层类比

本文的核心设计哲学是将 OS 中成熟的内存管理技术迁移到 GPU 上的 KV cache 管理：

- 物理页 → KV block
- 虚拟地址空间 → 序列的逻辑 KV 位置
- 页表 → block table
- 按需分页 (demand paging) → 按需分配 block
- Copy-on-Write → beam search / parallel sampling 的共享机制
- 页面换出 (swap out) → KV block 从 GPU 交换到 CPU

9. 相关工作

通用模型服务系统：Clipper、TensorFlow Serving、Triton 等关注批处理和路由，但没有处理 LLM 特有的 KV cache 内存问题。

Transformer 特定优化：FasterTransformer (NVIDIA) 关注 kernel 优化；Orca 关注迭代级调度；FlexGen 关注离线场景的 offloading。这些都没有解决 KV cache 碎片问题。

内存优化：训练中的激活重计算 (checkpointing)、混合精度等。本文是首次将 OS 分页思想应用于推理阶段 KV cache 管理的工作。

10. 结论

本文提出了 PagedAttention —— 一种将 KV cache 组织为非连续物理块、通过 block table 间接寻址的注意力算法，以及基于此算法的 vLLM 服务系统。通过消除内部/外部碎片、支持 block 级内存共享 (CoW)，vLLM 在各种模型和工作负载下均大幅超越先前最佳系统。我们相信这种将 OS 经典技术引入 ML 系统的思路对未来的系统设计具有启发意义。

译注

本翻译基于 [arXiv:2309.06180v1](https://arxiv.org/abs/2309.06180v1) 全文。论文中的图表 (Figure 1-11) 请参考原始 PDF。翻译过程中保留了核心术语的英文原文 (如 PagedAttention、KV cache、block table 等)，以便与代码和其他技术文档对照。

原文链接：<https://arxiv.org/abs/2309.06180>

vLLM 仓库：<https://github.com/vllm-project/vllm>

译者注：本翻译由 AI 辅助完成，供学习参考。